

RELATORIO DE AUDITORIA DE SEGURANCA

MissionLive V1

Data da auditoria: 2026-08-31

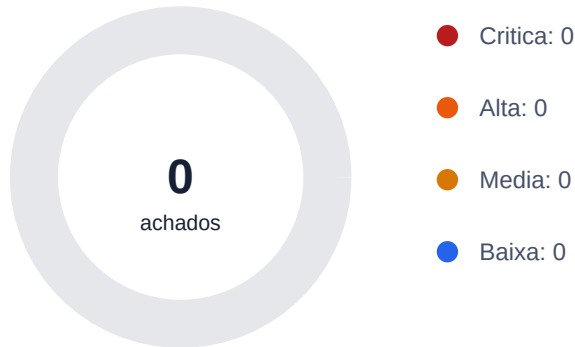
Escopo: API NestJS, frontend React, Prisma schema/migrations e configuracao segura

Resumo executivo

A auditoria percorreu os controllers reais da API, os services de dominio, o schema/migrations Prisma, a configuracao de ambiente e o bundle-fonte React. Nao foram confirmados achados de segurança nas cinco categorias avaliadas.

Foram encontrados 37 handlers HTTP. Todos os handlers sensíveis usam AuthGuard e os services resolvem o escopo por owner, membership direta ou membership de equipe antes de operar sobre o recurso.

Totais por severidade



Achados confirmados: 0 criticos, 0 altos, 0 medios e 0 baixos.

Distribuicao por categoria



Todas as categorias ficaram sem achados confirmados. As barras zeradas representam resultado da revisao, nao ausencia de cobertura.

Pontos fortes comprovados

Arquivo e linha	Evidencia
apps/api/src/auth/auth.guard.ts:15	const session = await this.prisma.session.findFirst({ where: { refreshTokenHash: hashToken(token), revokedAt: null, expiresAt: { gt: new Date() } }, include: { user: true } });
apps/api/src/goals/goals.service.ts:11	private accessWhere(userId: string, goalId?: string) { return { ...(goalId ? { id: goalId } : {}), OR: [{ ownerId: userId }, { members: { some: { userId } } }, { team: { ownerId: userId } }, { team: { members: { some: { userId } } } }] }; }
apps/api/src/main.ts:1	import { ValidationPipe } from '@nestjs/common';
apps/api/src/invites/invites.service.ts:15	const invite = await this.prisma.inviteLink.create({ data: { tokenHash: hashToken(token), targetType: dto.targetType, goalId, teamId, role: 'viewer', createdByUserId: userId, expiresAt } });

Achados detalhados

Nenhum achado real foi confirmado nesta auditoria. Em particular: não há consulta de Goal sem escopo de acesso em handlers expostos; a UI não é usada como controle de permissão; tokens de convite e refresh tokens são persistidos como hash; e não foram encontrados usos de `dangerouslySetInnerHTML`, `eval` ou `new Function` no frontend.

Pontos fracos e condições

- Validação de migration limpa contra PostgreSQL depende de URL/credencial SCRAM externa.
- Rate limiting de login é por processo; antes de escalar horizontalmente deve usar armazenamento compartilhado.

Recomendações P1/P2/P3

P1 - Nenhuma recomendação de correção urgente: não foram confirmados achados críticos ou altos.

P2 - Substituir o rate limiting em memória por armazenamento compartilhado antes de executar várias réplicas da API. Fornecer credencial PostgreSQL e executar o script de validação limpa antes da produção.

P3 - Adicionar testes de concorrência de aceite de convite contra PostgreSQL e completar o fluxo autenticado de vinculação de uma conta Google existente.

ISSUES PARA O GITHUB

Issue 1 - Rate limiting compartilhado antes do scale-out

Contexto: o limite de login atual é seguro para uma instância, mas o contador é local ao processo.

Problema: réplicas independentes podem aceitar mais tentativas agregadas do que o limite pretendido.

Acceptance criteria: usar store compartilhado com TTL; preservar mensagens sem revelar existência de conta; cobrir 429 em testes multi-instância; não logar credenciais.

Issue 2 - Validação PostgreSQL de pré-produção

Contexto: o script `scripts/validate-postgres.ps1` gera migration limpa do schema atual e aplica em banco informado.

Acceptance criteria: configurar `POSTGRES_DATABASE_URL` em ambiente seguro; executar migration do zero; rodar suite backend; registrar tipos, constraints, índices e concorrência de convites; remover credenciais do ambiente após o teste.

Critérios de aceite da auditoria

A auditoria cobre isolamento, autorização no backend, IDOR, segredos e XSS/input; reporta somente achados evidenciados; registra arquivos/linhas para pontos fortes; gera este PDF com A4, margens aproximadas de 2 cm, cabeçalho, rodapé, número de página, gráficos por severidade e categoria; e mantém este script regenerável.